

generate a value, and then use that value to determine what generator to use next. For this we need `flatMap`, which lets one generator depend on another.

**EXERCISE 8.6**

Implement `flatMap`, and then use it to implement this more dynamic version of `listOfN`. Put `flatMap` and `listOfN` in the `Gen` class.

```
def flatMap[B](f: A => Gen[B]): Gen[B]
def listOfN(size: Gen[Int]): Gen[List[A]]
```

**EXERCISE 8.7**

Implement `union`, for combining two generators of the same type into one, by pulling values from each generator with equal likelihood.

```
def union[A](g1: Gen[A], g2: Gen[A]): Gen[A]
```

**EXERCISE 8.8**

Implement `weighted`, a version of `union` that accepts a weight for each `Gen` and generates values from each `Gen` with probability proportional to its weight.

```
def weighted[A](g1: (Gen[A], Double), g2: (Gen[A], Double)): Gen[A]
```

8.2.5 Refining the *Prop* data type

Now that we know more about our representation of generators, let's return to our definition of `Prop`. Our `Gen` representation has revealed information about the requirements for `Prop`. Our current definition of `Prop` looks like this:

```
trait Prop {
  def check: Either[(FailedCase, SuccessCount), SuccessCount]
}
```

`Prop` is nothing more than a non-strict `Either`. But it's missing some information. We have the number of successful test cases in `SuccessCount`, but we haven't specified how many test cases to examine before we consider the property to have *passed* the test. We could certainly hardcode something, but it would be better to abstract over this dependency:

```
type TestCases = Int
type Result = Either[(FailedCase, SuccessCount), SuccessCount]
case class Prop(run: TestCases => Result)
```

Also, we're recording the number of successful tests on both sides of that `Either`. But when a property passes, it's implied that the number of passed tests will be equal to the argument to `run`. So the caller of `run` learns nothing new by being told the success count. Since we don't currently need any information in the `Right` case of that `Either`, we can turn it into an `Option`:

```
type Result = Option[(FailedCase, SuccessCount)]
case class Prop(run: TestCases => Result)
```

This seems a little weird, since `None` will mean that all tests succeeded and the property passed and `Some` will indicate a failure. Until now, we've only used the `None` case of `Option` to indicate failure. But in this case we're using it to represent the *absence* of a failure. That's a perfectly legitimate use for `Option`, but its intent isn't very clear. So let's make a new data type, equivalent to `Option[(FailedCase, SuccessCount)]`, that shows our intent very clearly.

Listing 8.2 Creating a Result data type

```
sealed trait Result {
  def isFalsified: Boolean
}
case object Passed extends Result {
  def isFalsified = false
}
case class Falsified(failure: FailedCase,
                    successes: SuccessCount) extends Result {
  def isFalsified = true
}
```

Indicates that all tests passed

Indicates that one of the test cases falsified the property

Is this now a sufficient representation of `Prop`? Let's take another look at `forall`. Can `forall` be implemented? Why not?

```
def forall[A](a: Gen[A])(f: A => Boolean): Prop
```

We can see that `forall` doesn't have enough information to return a `Prop`. Besides the number of test cases to try, `Prop.run` must have all the information needed to generate test cases. If it needs to generate random test cases using our current representation of `Gen`, it's going to need an `RNG`. Let's go ahead and propagate that dependency to `Prop`:

```
case class Prop(run: (TestCases, RNG) => Result)
```

If we think of other dependencies that it might need, besides the number of test cases and the source of randomness, we can just add these as extra parameters to `Prop.run` later.

We now have enough information to actually implement `forall`. Here's a simple implementation.